UAH Mars Drone ASW
Architecture Design Document

Index

5 Software Design

5.1 Software Static Architecture

5.2 Software Dynamic Architecture

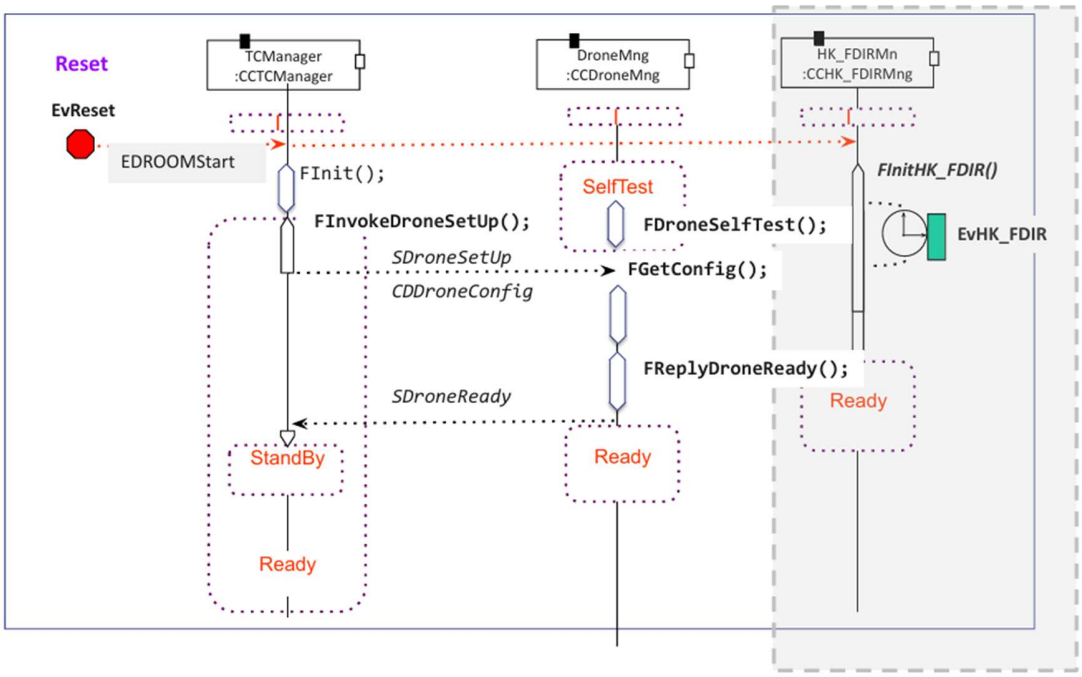
5.2.1 Computational Model

5.2.2 Software Behaviour

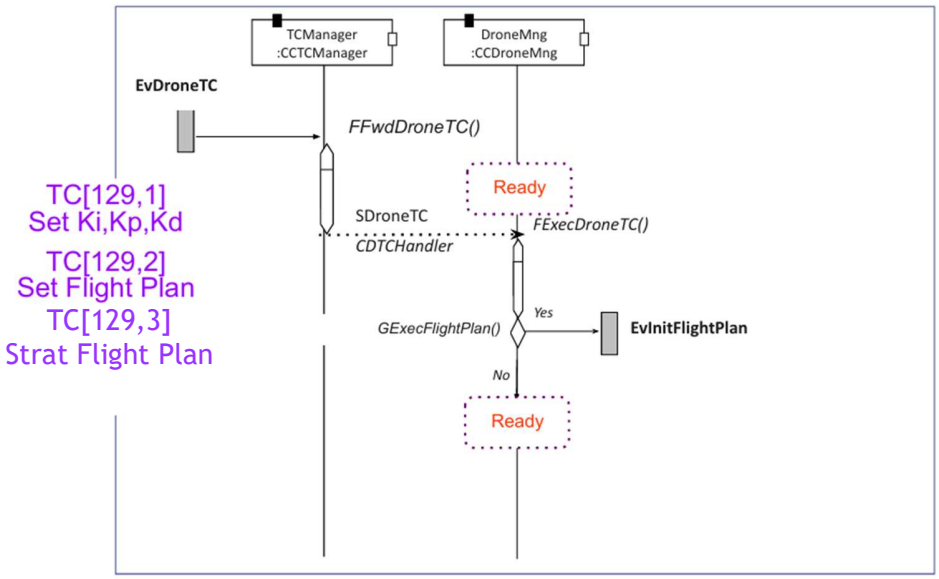
Scenarios

Event	Description
EvReset (external)	System power-on
EvRxTC (external IRQ)	TC reception
EvAcceptedTC (internal)	TC acceptation
EvRejectedTC (internal)	TC rejection
EvToReboot (internal)	System Reboot
EvHK_FDIR (timing)	Periodic HK and FDIR Mng
EvHK_FDIR_TC (internal)	HK_FDIR TC Execution
EvPrioTC (internal)	Priority TC execution
EvBKGTC (internal)	Background TC execution
EvTCEventAction (internal)	Event-based action TC execution
EvDroneTC (internal)	Drone TC execution
EvInitFlightPlan (internal)	Start of the Drone Flight Plan
EvFlightCtrl (timing)	Periodic flight plan status check
EvDroneTCInFligt (internal)	Drone TC execution during flight
EvFlightPlanDone (internal)	Back to ready after completing the plan

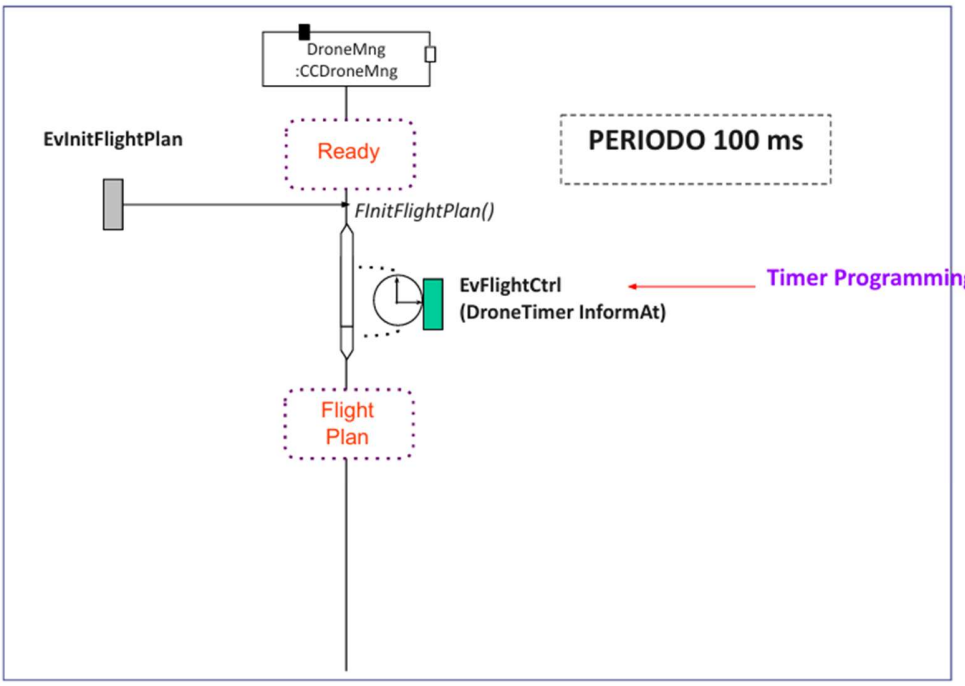
EvReset



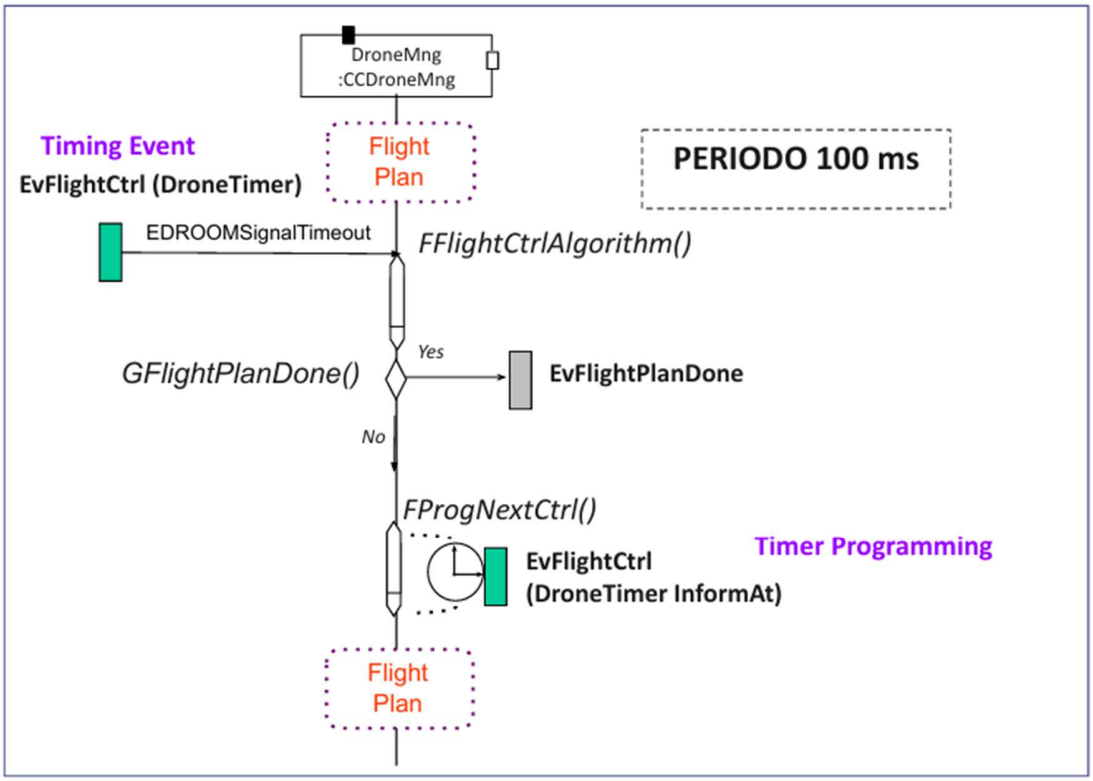
EvDroneTC



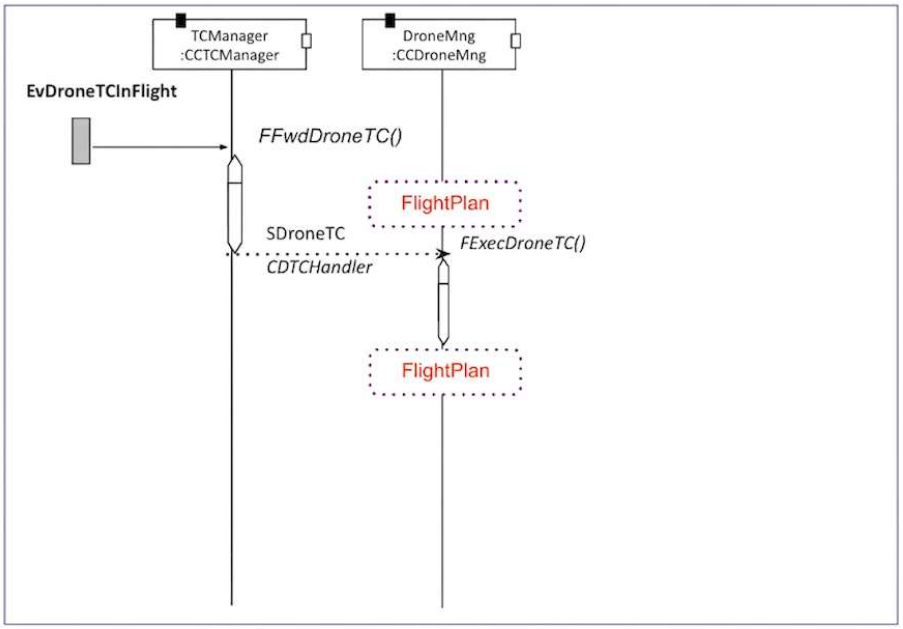
EvInitFlightPlan



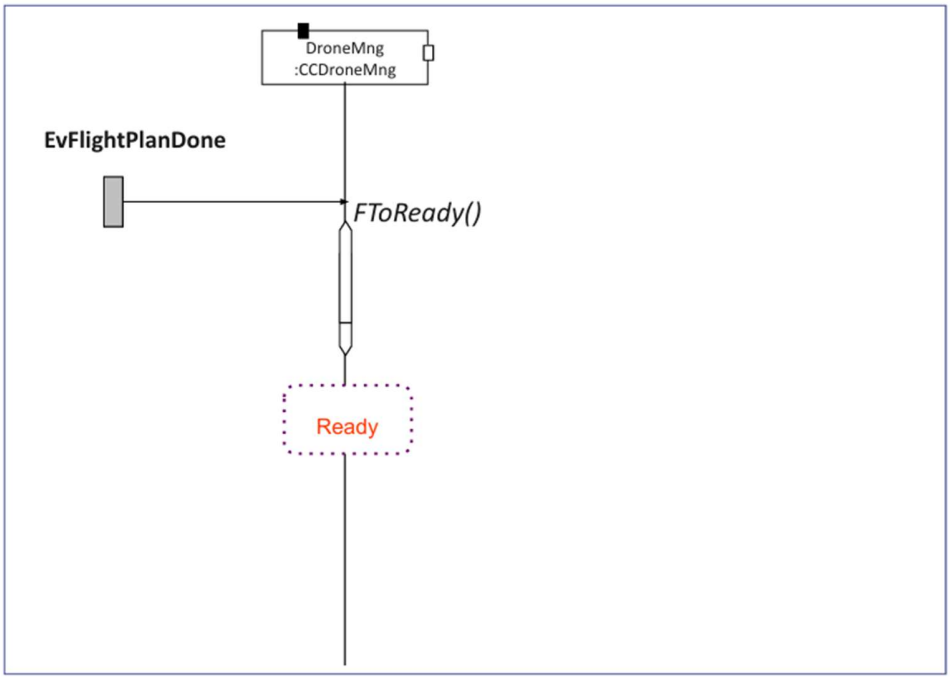
EvFlightCtrl



EvDroneTCInFligt



EvFlightPlanDone



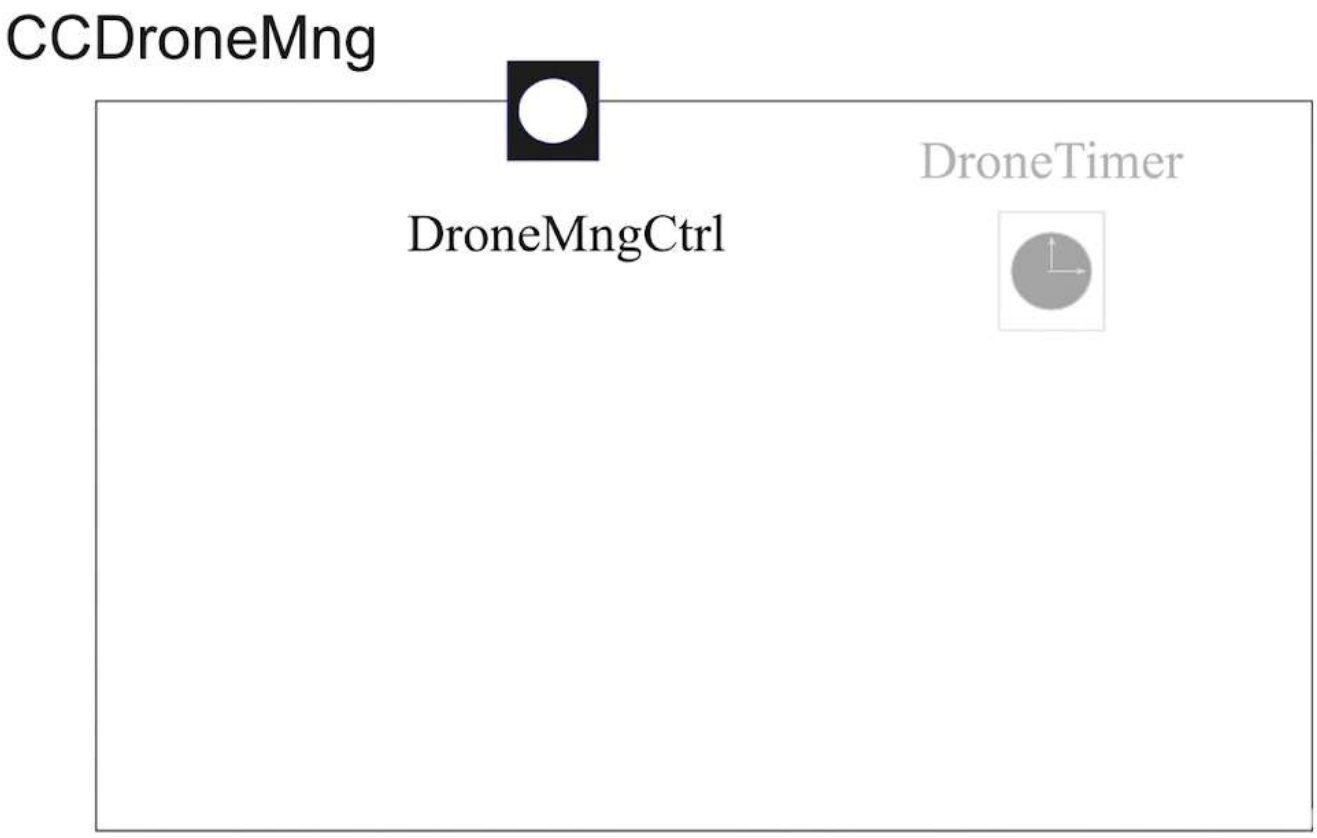
5.3 Real-Time Requirements

5.4 Software Components Design

5.4.1 CCDroneMng

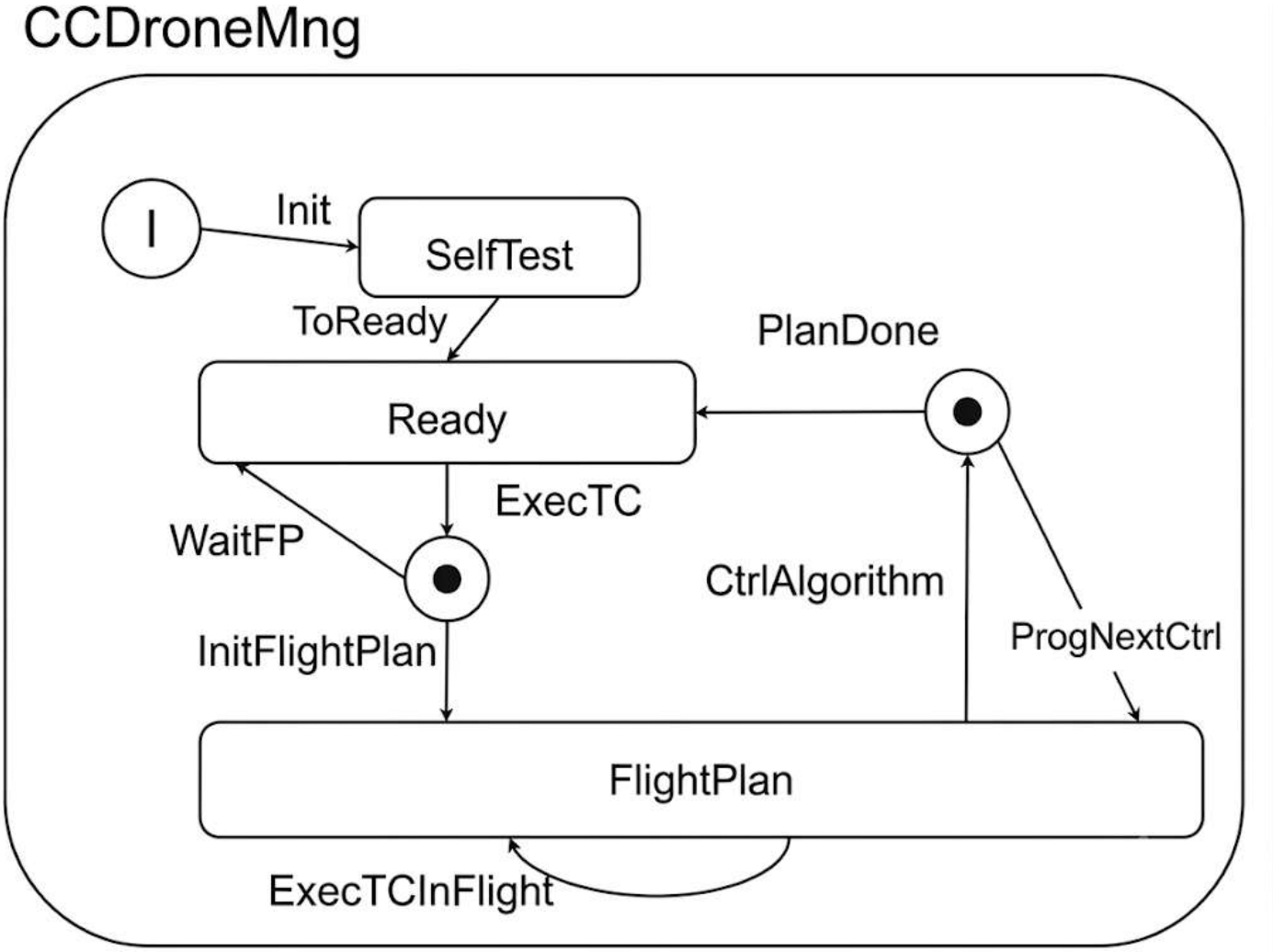
5.4.1.1 Interfaces

CCDroneMng **Interfaces**

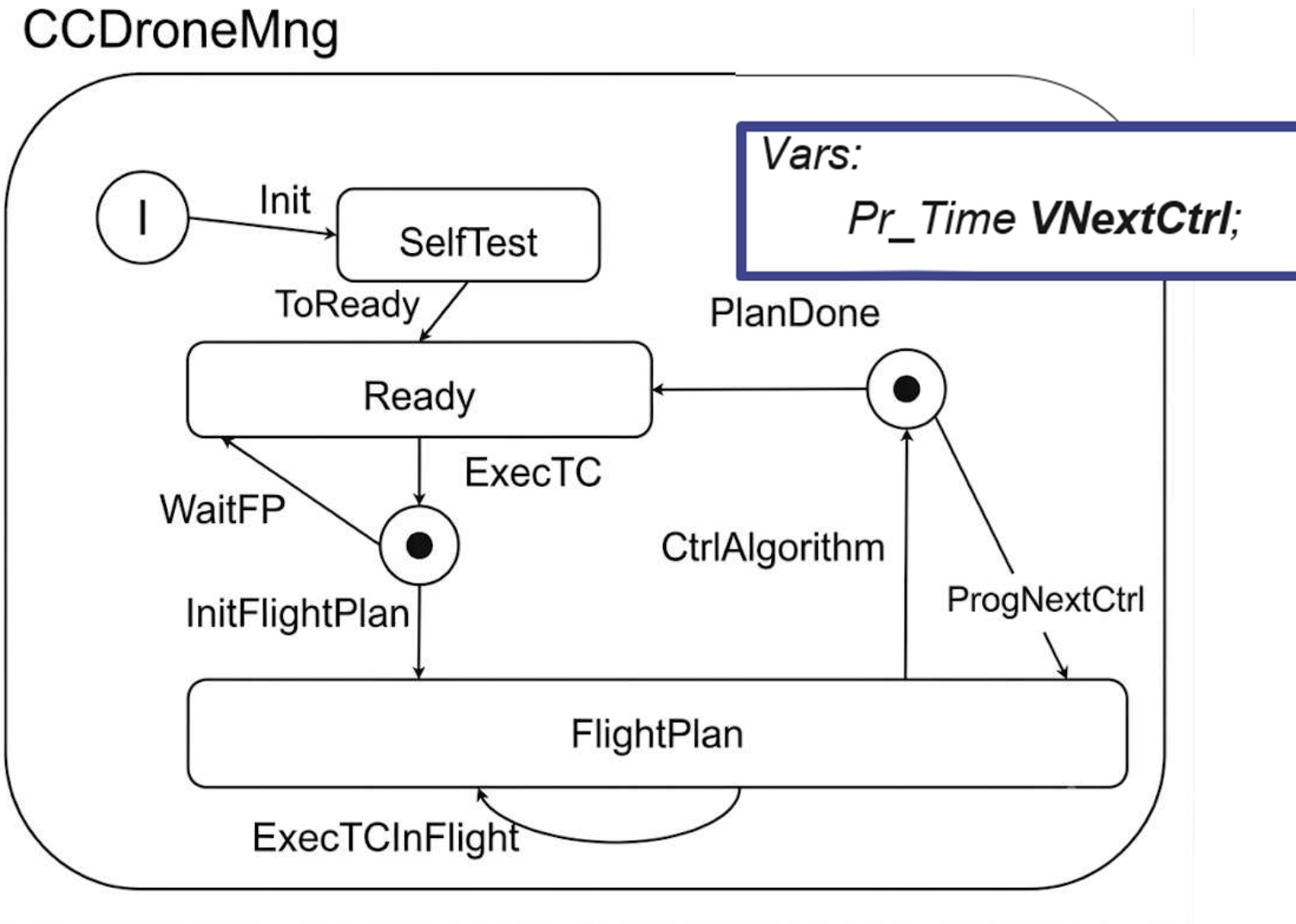


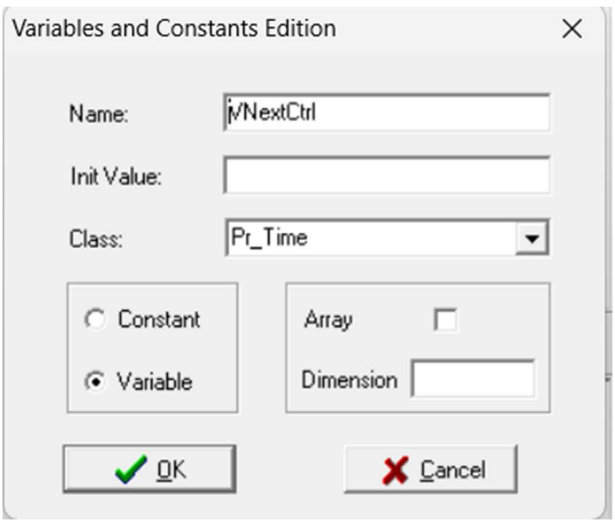
5.4.1.1 Behaviour

Diagrama de estados

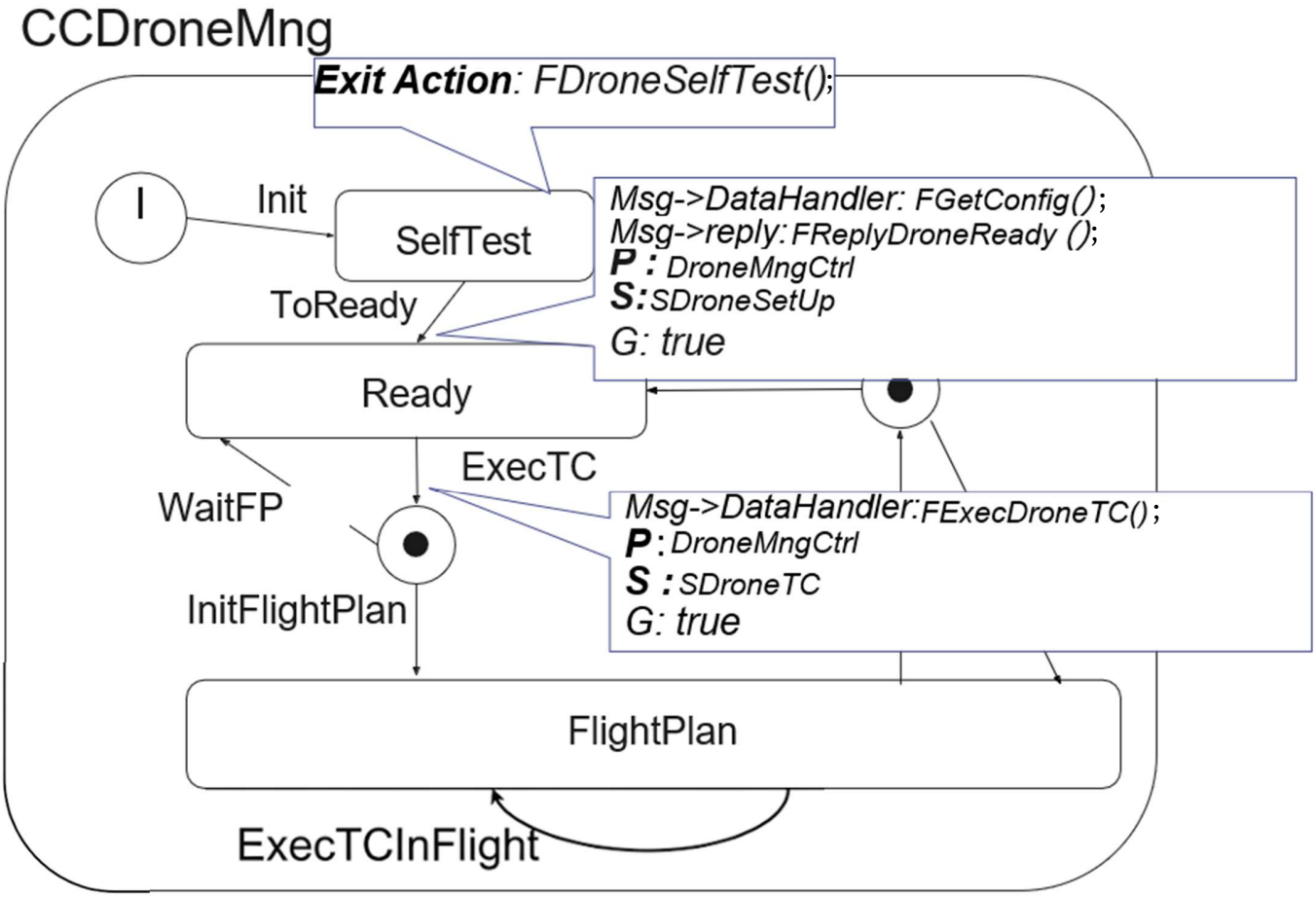


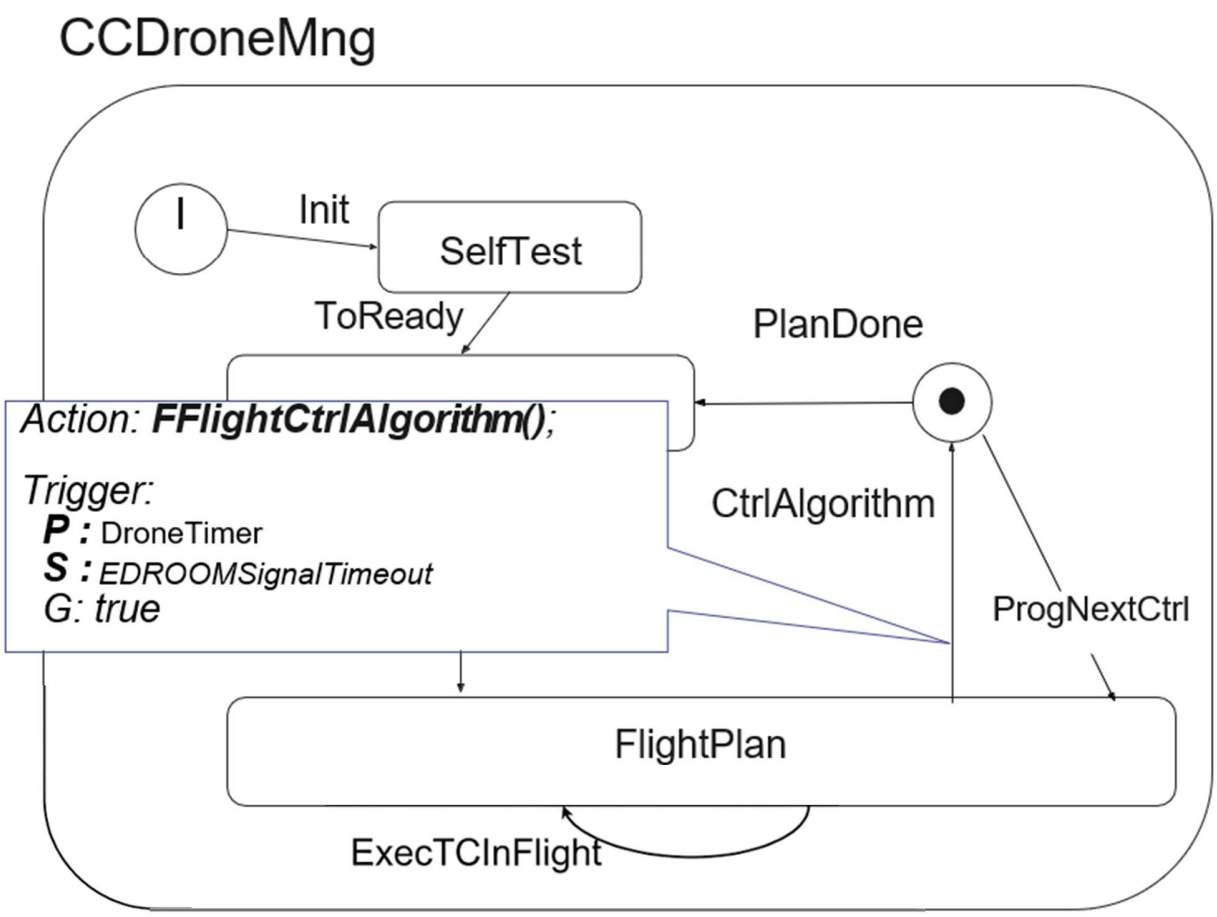
Variables de la clase CCDroneMng

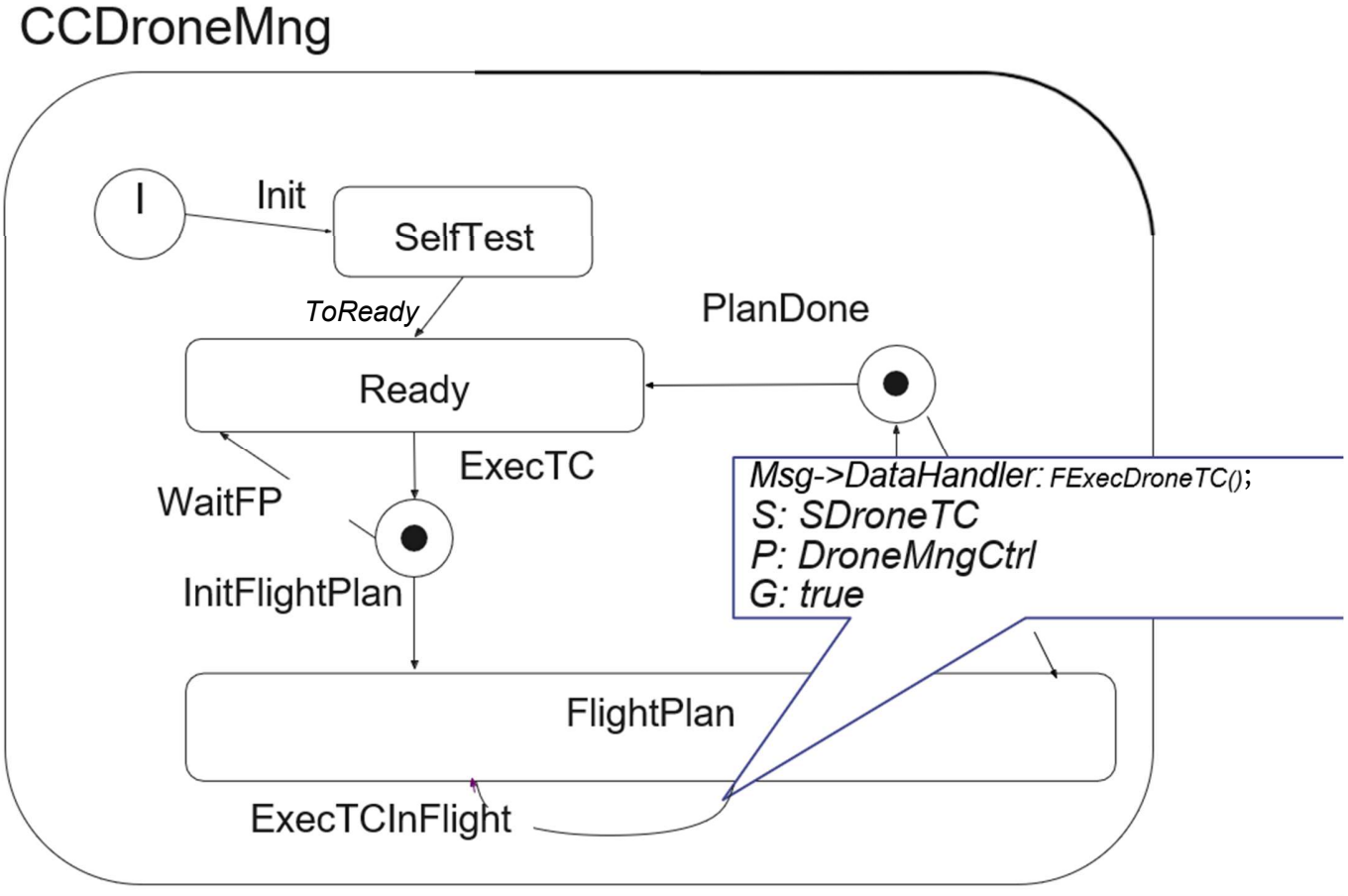




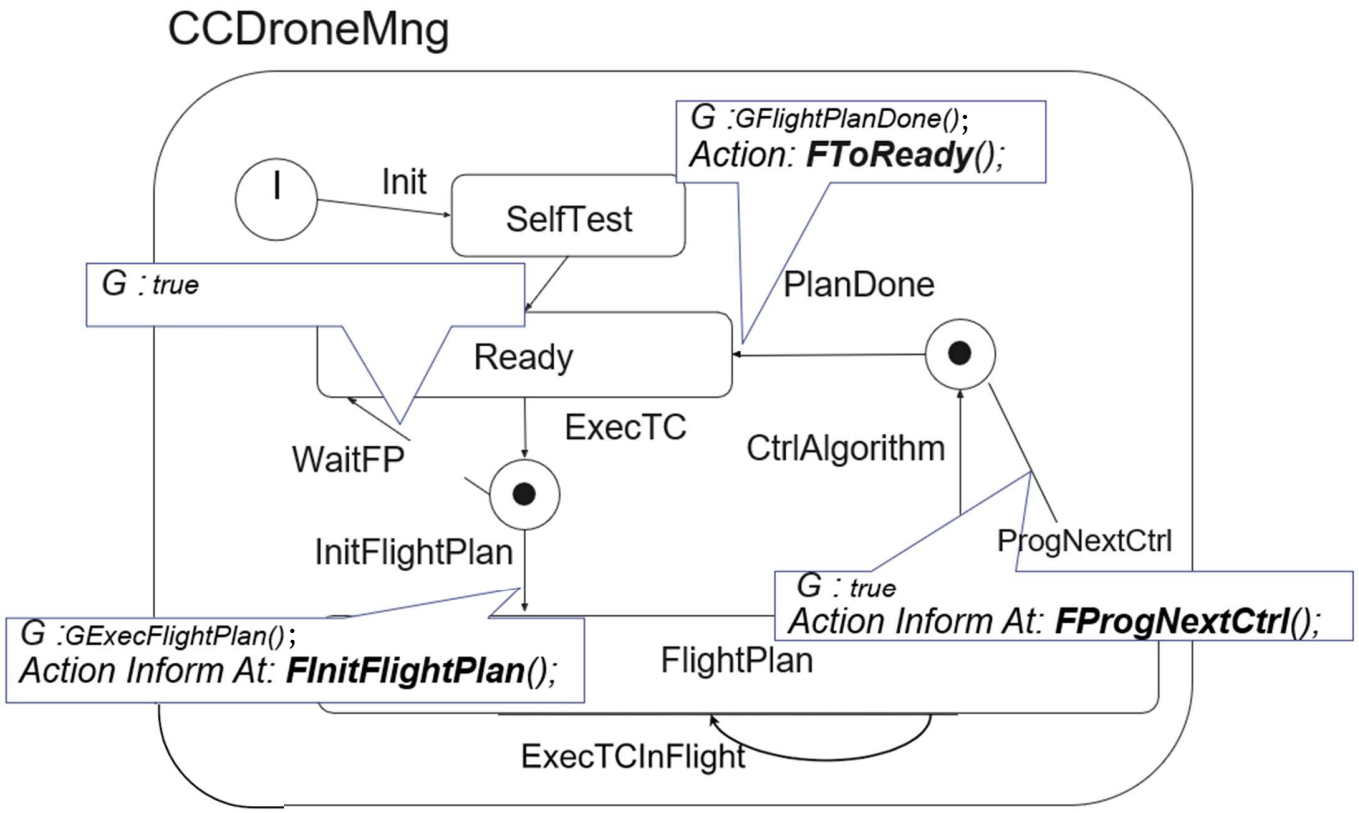
Transiciones, acciones y condición de disparo:




CCDroneMng
Init
SelfTest
ToReady
PlanDone
Action: FFlightCtrlAlgorithm();
Trigger:
P : DroneTimer
S : EDROOMSignalTimeout
G: true
CtrlAlgorithm
ProgNextCtrl
FlightPlan
ExecTCInFlight


CCDroneMng
Init
SelfTest
ToReady
PlanDone
Ready
ExecTC
WaitFP
InitFlightPlan
Msg->DataHandler: FExecDroneTC();
S: SDroneTC
P: DroneMngCtrl
G: true
FlightPlan
ExecTCInFlight

Guardas y acciones asociadas:



Código de las acciones y guardas:

```
void FDroneSelfTest(){

    pus_service129_drone_selftest();


}
```

```
void FReplyDroneReady(){

    pus_service129_drone_ready();

    Msg->Reply(SDroneReady);
}
```

```
void FGetConfig(){

  CDDroneConfig & varSDroneSetUp = *(CDDroneConfig *) Msg->data;

    pus_service129_setup(varSDroneSetUp);

}
```

```
void FExecDroneTC(){

  CDTCHandler & varSDroneTC = *(CDTCHandler *) Msg->data;


    varSDroneTC.ExecDroneTC();
}
```

```
bool GExecFlightPlan(){

    return pus_service129_flight_plan_ready();


}
```

```
bool GFlightPlanDone(){

    return pus_service129_flight_plan_done();


}
```

PERÍODO 100 ms

```
void FInitFlightPlan(){
    Pr_Time time;

    time.GetTime(); // Get current monotonic time
    time+=Pr_Time(0,100000); // Add X sec + Y microsec
    VNextCtrl=time;

    pus_service129_init_flight_plan();

    DroneTimer.InformAt(time);

}
```

PERÍODO 100 ms

```
void FProgNextCtrl(){
    Pr_Time time;

    VNextCtrl+=Pr_Time(0,100000);
    // Add X sec + Y microsec
    time=VNextCtrl;

    DroneTimer.InformAt(time);


}
```

```
void FFlightCtrlAlgorithm(){

  pus_service129_flight_ctrl_algorithm();


}
```

```
void FToReady(){

   pus_service129_drone_ready();

}
```